

CPSC 340: Machine Learning and Data Mining

Neural Networks

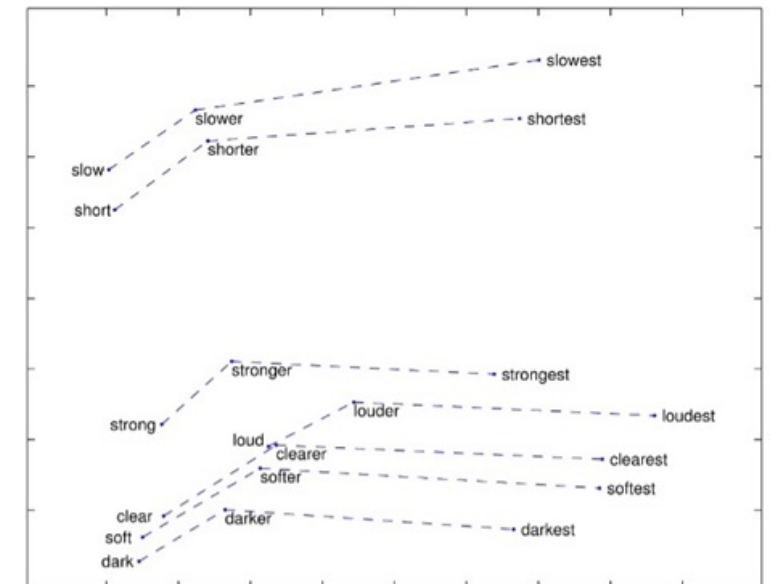
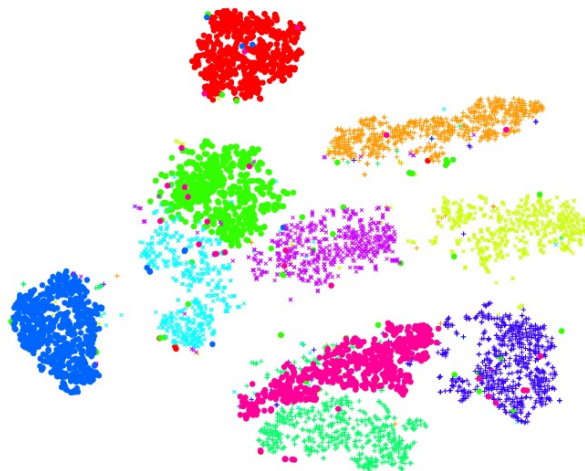
Admin

- It's nice to be back!
- A5 due Friday (Nov. 21)
- A6 already released in you want to start right away (you can already do the PCA/recommender systems part)
- Will do a lecture / discussion on Transformers / RLHF (probably Dec 3, but might reorder things a bit). Let me know if there are specific things you are curious about! (email/piazza)

Last Time: Multi-Dimensional Scaling

- Recommender systems: two approaches
- Multi-dimensional scaling (MDS):
 - Non-parametric latent-factor model: directly optimizes the z_i into low dimension
 - T-SNE tends to visualize clusters and manifold structures.
 - Word2vec gives continuous alternative to bag of words.

visualization of
high dimension
into low dimension



End of Part 4: Key Concepts

- We discussed **linear latent-factor models:**

$$f(W, Z) = \sum_{i=1}^n \sum_{j=1}^d (\langle w_j^i, z_j \rangle - x_{ij})^2$$

$$= \sum_{i=1}^n \|W^T z_i - x_i\|^2$$

$$= \|ZW - X\|_F^2$$

latent-factor
models:

PCA part

- Represent 'X' as linear combination of **latent factors 'w_c'**.
 - Latent features 'z_i' give a lower-dimensional version of each 'x_i'.
 - When k=1, finds **direction that minimizes squared orthogonal distance**.
- Applications:
 - Dimensionality reduction, data compression, features for linear models, visualization, outlier detection, factor discovery, filling in missing entries.

End of Part 4: Key Concepts

- Principal component analysis (PCA):
 - Often uses orthogonal factors and fits them sequentially (via SVD).
 - Or uses non-orthogonal factors and fits with SGD.
- Generalizations of PCA using ideas from linear models:
 - Binary PCA, robust PCA, regularized PCA, sparse PCA, NMF.
- Recommender systems:
 - “Content-based filtering” is usually supervised learning approach.
 - Collaborative-filtering only uses ratings.
- Matrix factorization approach to collaborative filtering.
 - Fits regularized PCA to available entries in matrix, to “fill in” other entries.

End of Part 4: Key Concepts

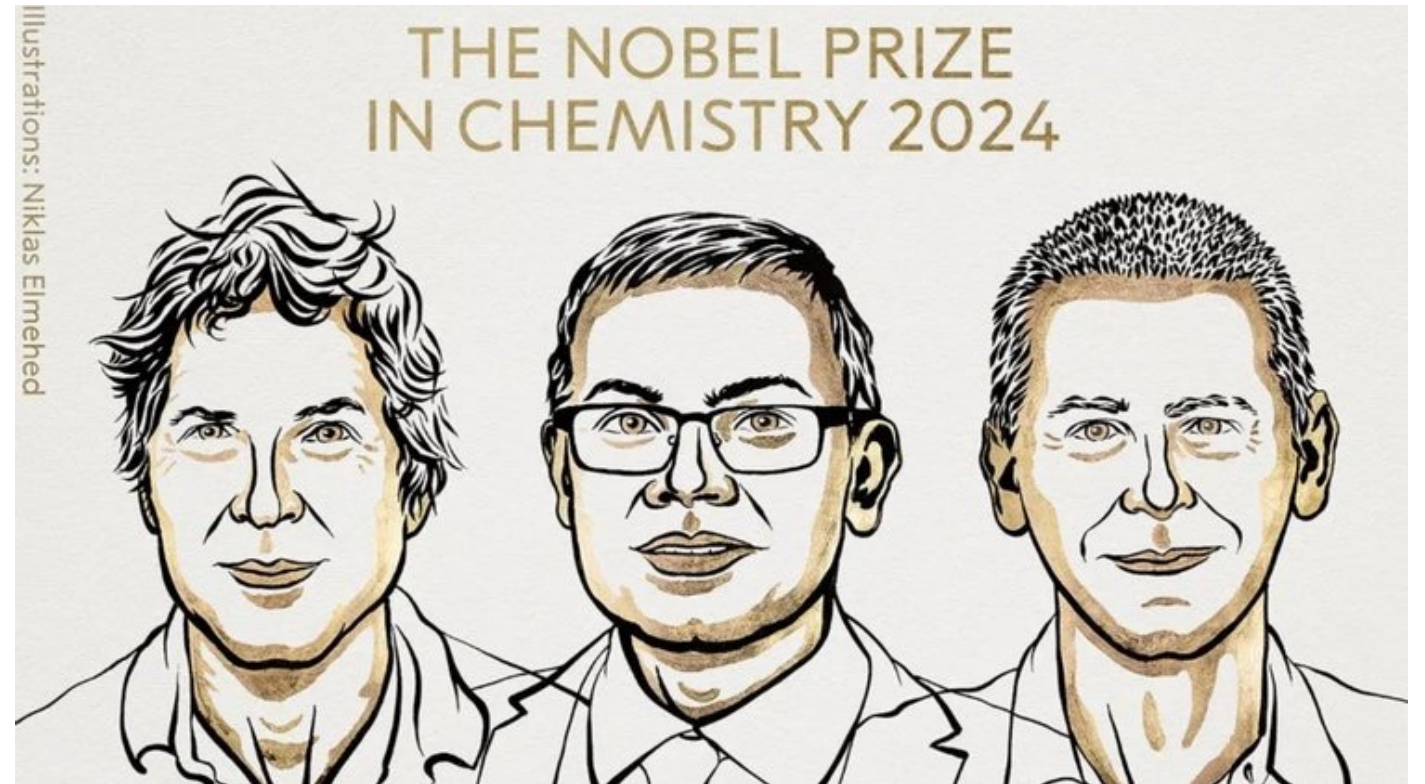
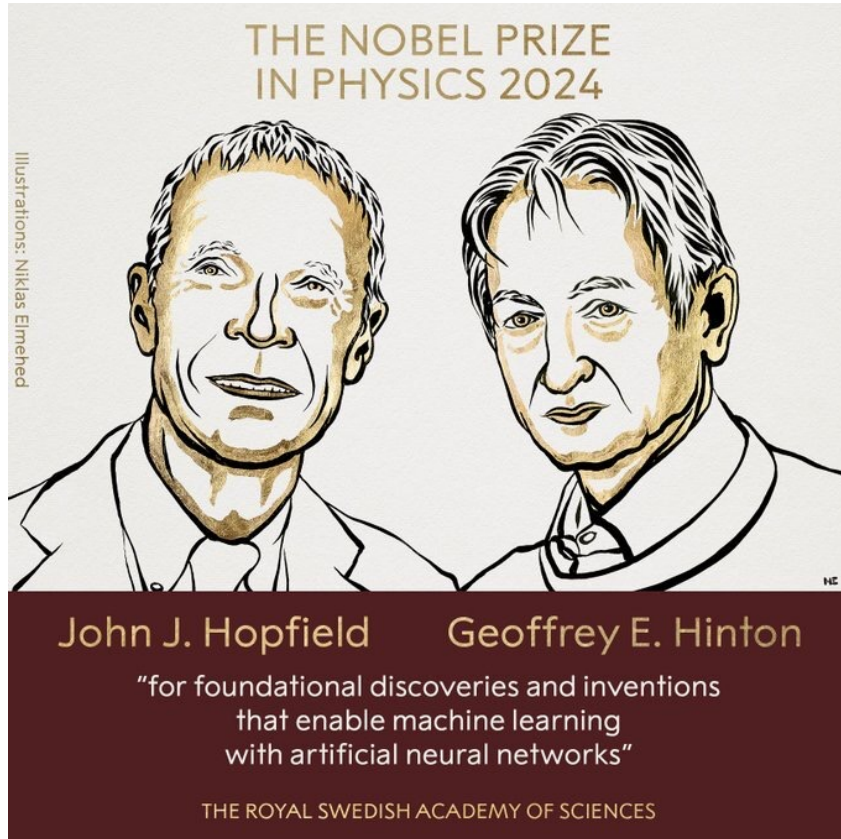
- We discussed **multi-dimensional scaling (MDS)**:
 - **Non-parametric** method for high-dimensional **data visualization**.
 - Tries to match distance/similarity in high-/low-dimensions.
 - “Gradient descent on scatterplot points”.
- Main **challenge in MDS methods is “crowding”** effect:
 - Methods focus on large distances and lose local structure.
- We discussed **t-SNE**:
 - MDS focusing on neighbour distances and not large distances.
- **Word2vec** is a recent MDS method giving better “word features”.

Next Topic: Neural Networks

Neural Network History

- Popularity of neural networks has come in waves over the years.
 - Currently, it is **one of the hottest topics in ML / AI, and even science.**
- Recent popularity due to **unprecedented performance** on some difficult tasks.
 - Speech recognition, natural language processing.
 - Computer vision.
 - Science.
- There are mainly due to big datasets, deep models, and tons of computation.
 - Plus more complex “architectures” (e.g. CNNs, LSTMs, ResNets, transformers)
- Lots of histories of the field online

Deep Learning Winning Nobel Prizes



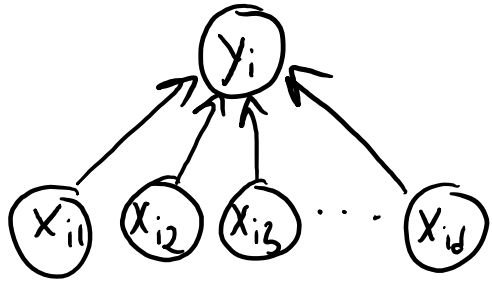
Neural Networks: Motivation

- Many domains **require non-linear transforms** of the features.
 - But, it **may not be obvious which transform** to use.
- **Neural network** models try to **learn good transformations**.
 - Optimize the “parameters of the feature transformations”.
 - And choose a class of features that help solve the classification/regression problems.
- We will first discuss the special case of “one hidden layer”.
 - Then we will move onto “deep learning” with uses multiple layers.

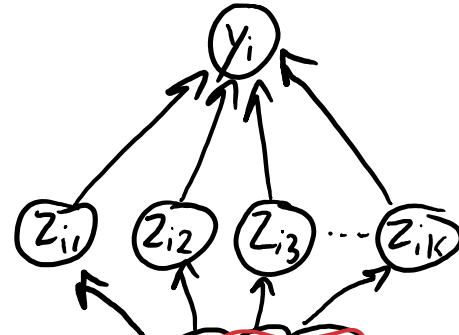
بسیار جالبی.

A Graphical Summary of CPSC 340 Parts 1-5

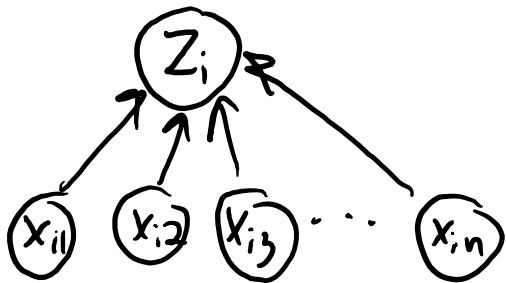
Part 1: "I have features x_i "



Part 3: change of basis

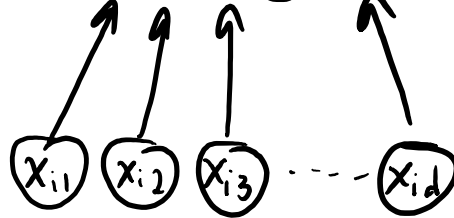


Part 2: "What is the group of x_i ?"



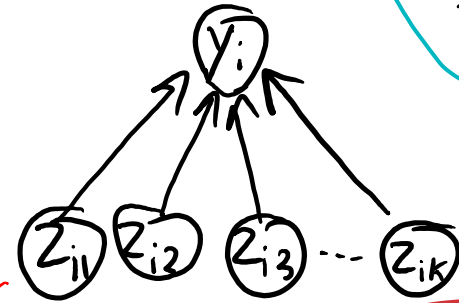
chunk of labels.

"I think this basis will work"

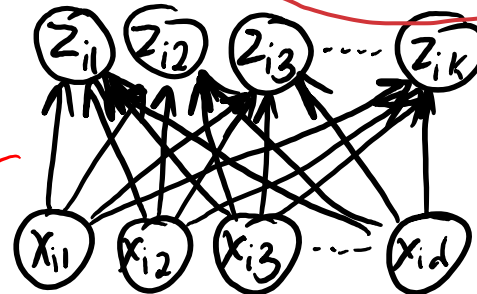


Trained separately

Part 4: basis from latent-factor model

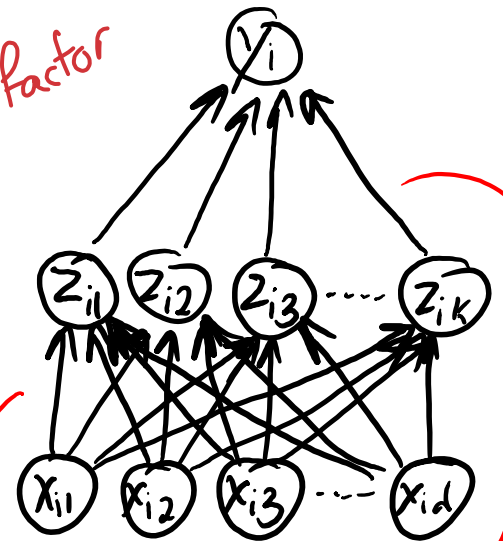


"PCA will give me good features"



"What are the 'parts' of x_i ?"

Part 5: Neural networks



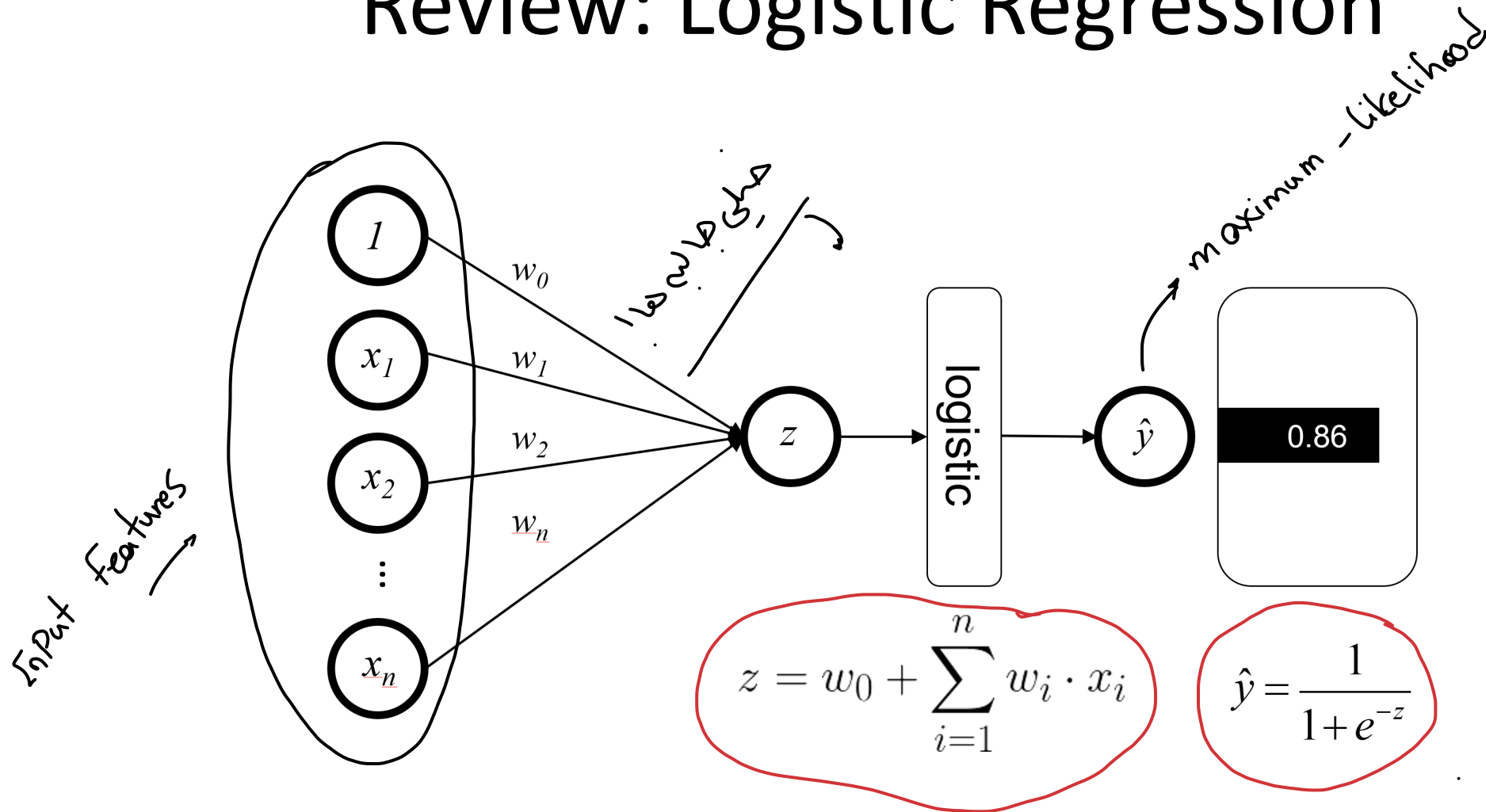
Learn features and classifier at the same time.

when we don't know which features

Why not just Neural Networks? *Ha Ha*

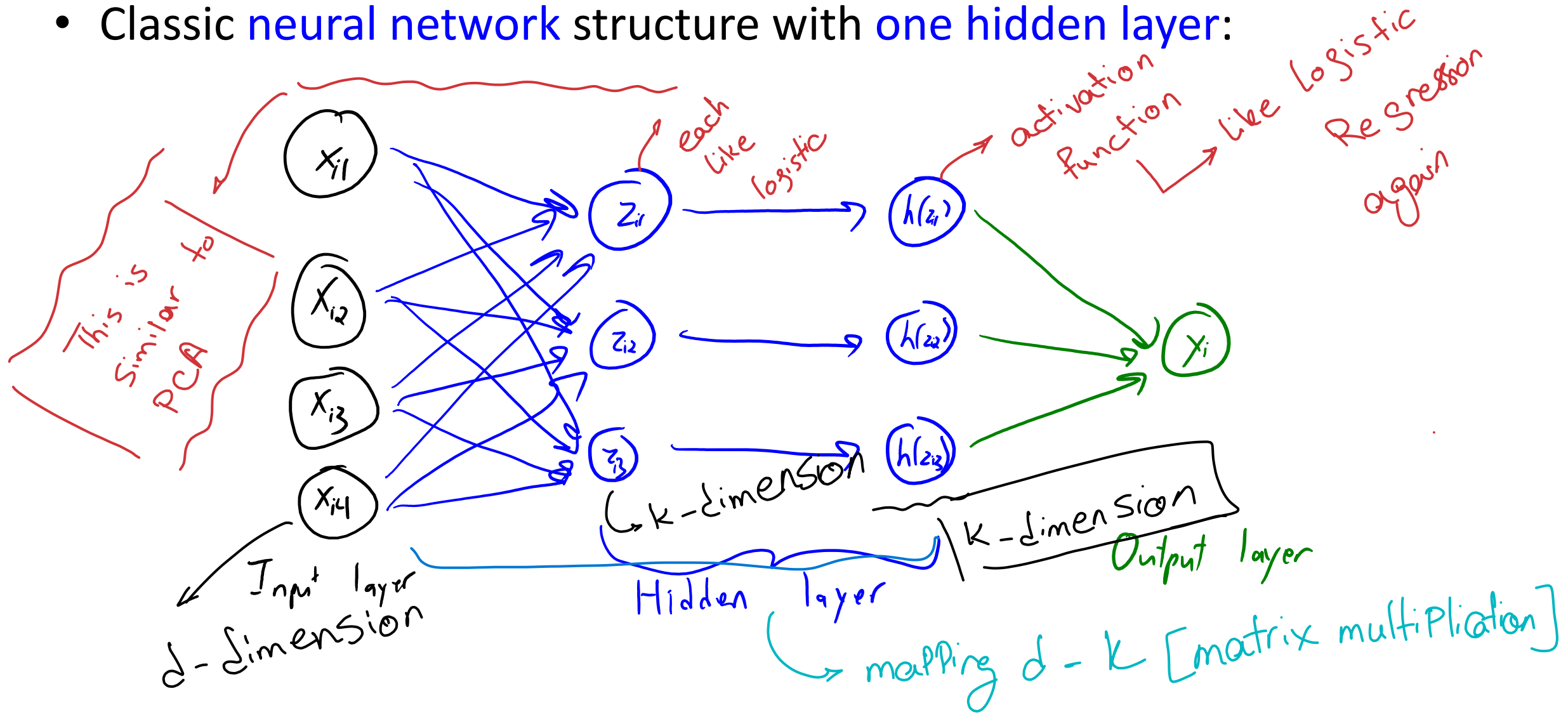
- Many applications **still rely on “traditional” models.**
 - Boosted trees and logistic regression with specific featurization still dominates in many applications.
- **Neural networks** directly build on, or get inspiration from, **“traditional” ML ideas we have seen.**
 - Losses, SGD, linear models, factor models, regularization, train/val/test split ...
 - Deep learning is basically **SGD** on complex parametrized functions that we can differentiate automatically! *stochastic gradient descent.*

Review: Logistic Regression



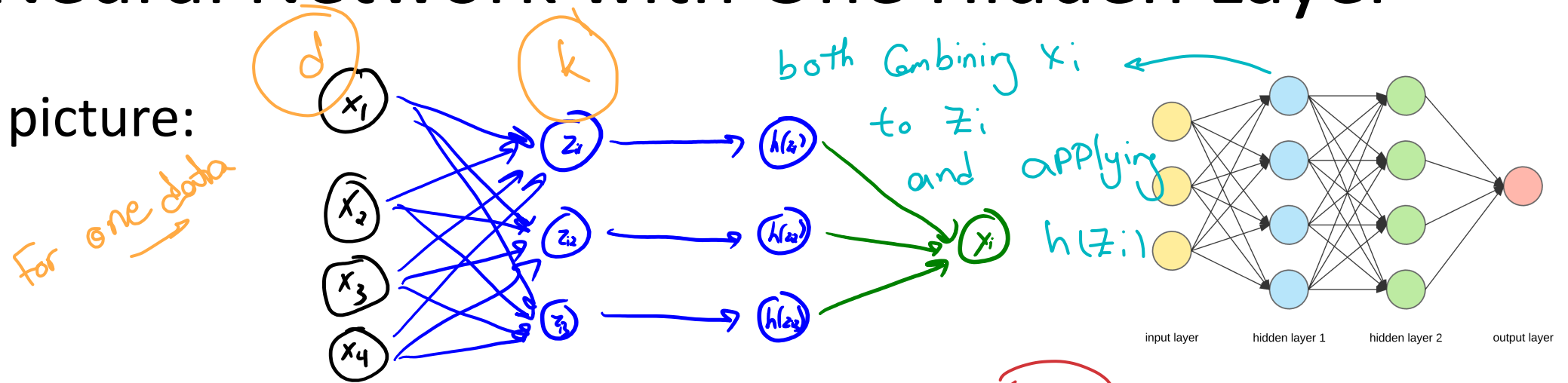
Neural Network with One Hidden Layer

- Classic **neural network** structure with **one hidden layer**:



Neural Network with One Hidden Layer

- As a picture:



- As a function:

$$\hat{y}_i = v^T h(W x_i)$$

Linear combination of "activations"

Non-linear transformation of each z_{ic}

logistic

i^{th} training example
 c^{th} z value

$k \times d$
" z_i ": linear combination of input

Matrix multiplication that map $d \rightarrow n$

$$W x_i = z_i$$

latent-factor

$$Z = \begin{bmatrix} z_1^T \\ z_2^T \\ \vdots \\ z_n^T \end{bmatrix}$$

$n \times k$

It is common to have $k \times d$

Neural Network with One Hidden Layer

- As a function:

$$\hat{y}_i = v^T h(W x_i)$$

$\hat{y}_i$: Linear combination of "activations"
 h : Non-linear transformation of each z_i
 $W x_i$: "z_i": linear combination of input
 h is 1 → 1
 sigmoid activation / ReLU

$k \times d + 1$ Parameters. For one row of data

- Parameters:** the "k times d" matrix ' W ', and length-k vector " v ".

- Using 'k' as number of "hidden units", the dimension are:

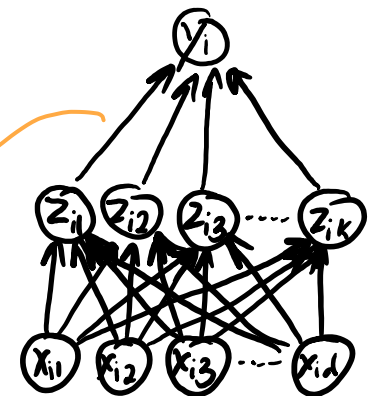
$$W = \begin{bmatrix} \text{---} & w_1^T & \text{---} \\ \text{---} & w_2^T & \text{---} \\ \vdots & \vdots & \vdots \\ \text{---} & w_k^T & \text{---} \end{bmatrix}$$

$k \times d$

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_k \end{bmatrix}$$

$k \times 1$

Parameters are the same as arrows



Neural Network with One Hidden Layer

- As a function:

$$\hat{y}_i = v^T h(W x_i)$$

Linear combination of "activations"

Non-linear transformation of each z_i

" z_i ": linear combination of input

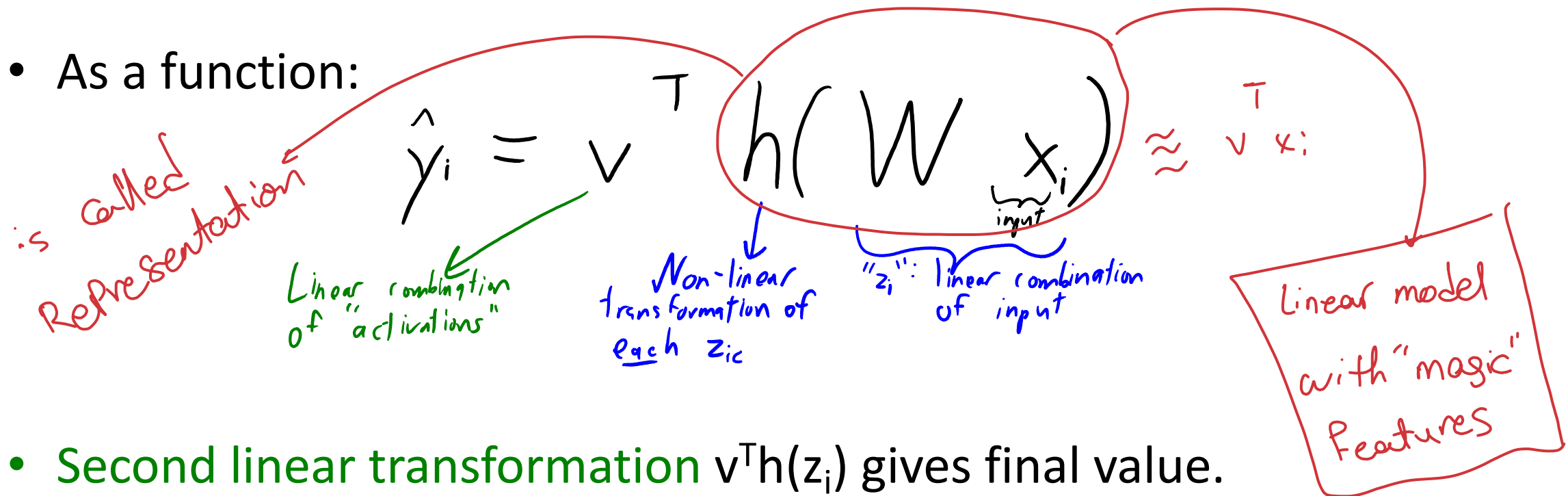
Here ∞

Love if ∞

- Linear transformation $z_i = Wx_i$ is like doing PCA.
 - Mixes together the features in a way that we learn.
- Non-linear transform 'h' is "often" sigmoid applied element-wise.
 - Without a non-linear transformation it degenerates to a linear model:
 - $\hat{y}_i = v^T(Wx_i) = (v^TW)x_i = w^Tx_i$ (if we set 'w' using $w = W^Tv$).

Neural Network with One Hidden Layer

- As a function:



- Second linear transformation $v^T h(z_i)$ gives final value.

– This is like using a **linear model with non-linear feature transformations**.

- But in this case we learned the features.

which is the main thing oo

- Cost of computing $\hat{y}_i$ above is **$O(kd)$** .

– $O(kd)$ to compute Wx_i , $O(k)$ to apply 'h', then $O(k)$ to multiply by 'v'.

Conceptual questions

Which are true about neural networks:

- A: Logistic regression can be considered as a simple neural network
- B: Neural networks automatically learn feature transformation to solve regression/classification tasks
- C: The nonlinear transformation acts on each element z_{ic}
- D: Without nonlinear transformation, neural networks can only extract linear features

Why Sigmoid as Non-Linear Transform?

- Consider setting 'h' to define **binary features z_i** using:

$$h(z_{ic}) = \begin{cases} 1 & \text{if } z_{ic} \geq 0 \\ 0 & \text{if } z_{ic} < 0 \end{cases}$$



- Each $h(z_i)$ can be viewed as binary feature.
 - “You either have this ‘part’ or you don’t have it.”
- We can make 2^k objects by all the possible “part combinations”.

} Not good since gradient is zero or unknown }

Motivation: Pixels vs. Parts

Motivation: Pixels vs. Parts

- We could represent other digits as different combinations of “parts”:

The diagram illustrates the representation of digits 3, 5, and 8 as combinations of basic parts (strokes). Each digit is shown on the left, followed by an equals sign and a sequence of parts in boxes. Green '+' signs indicate positive contributions, and red '+' signs indicate negative contributions (removal of a part).

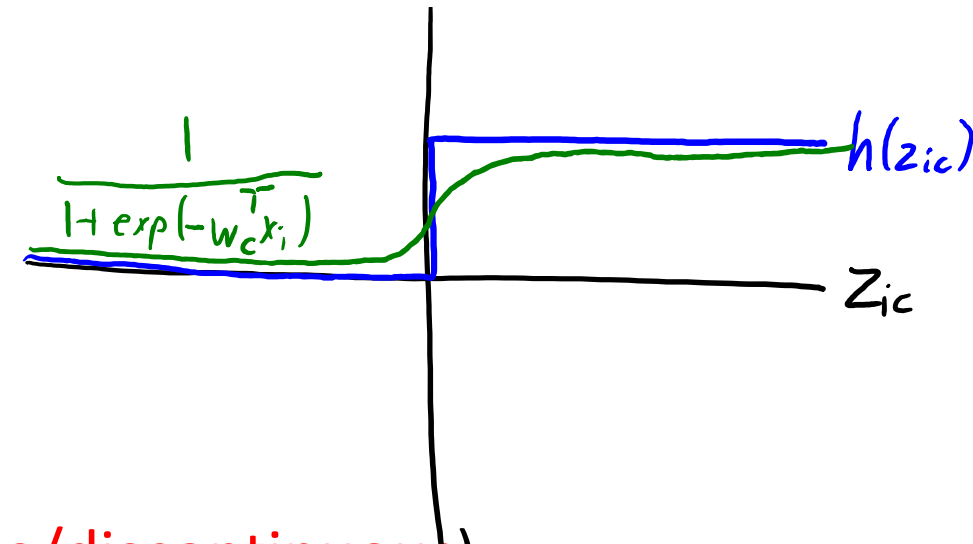
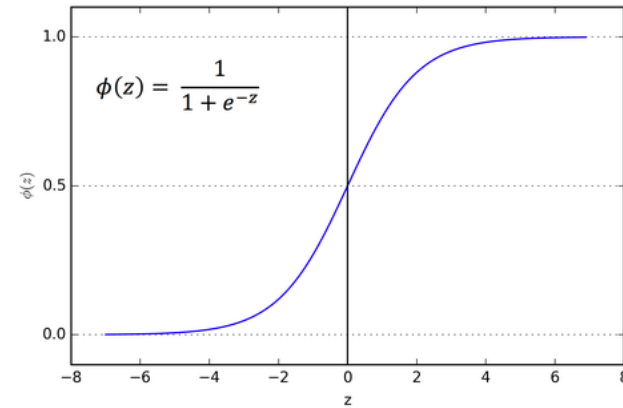
- Digit 3:** Represented as a horizontal stroke (+), a curved stroke (+), a horizontal stroke (+), a vertical stroke (+), a curved stroke (+), and a horizontal stroke (+).
- Digit 5:** Represented as a horizontal stroke (+), a curved stroke (+), a horizontal stroke (+), a vertical stroke (+), a curved stroke (+), and a horizontal stroke (+).
- Digit 8:** Represented as a horizontal stroke (+), a curved stroke (+), a horizontal stroke (+), a vertical stroke (+), a curved stroke (+), and a horizontal stroke (+).

Why Sigmoid as Non-Linear Transform?

- Consider setting 'h' to define **binary features** z_i using

$$h(z_{ic}) = \begin{cases} 1 & \text{if } z_{ic} \geq 0 \\ 0 & \text{if } z_{ic} < 0 \end{cases}$$

- Each $h(z_i)$ can be viewed as binary feature.
 - “You either have this ‘part’ or you don’t have it.”
- But this is hard to optimize (**non-differentiable/discontinuous**).
- **Sigmoid is a smooth approximation** to these binary features.
 - Allows you to train the model using gradient descent or SGD.



Universal Approximation with One Hidden Layer

- Classic choice of “activation” function ‘h’ is the sigmoid function.
 - With enough hidden “units”, this is a “universal approximator”.
 - Any continuous function can be approximated arbitrarily well (on bounded domain).
 - But this result is for a non-parametric setting of the parameters:
 - The number of hidden “units” must be a function of ‘n’.
 - A fixed-size network is not a universal approximator.
 - Other universal approximators (always non-parametric):
 - K-nearest neighbours.
 - Need to have ‘k’ depending on ‘n’.
 - Linear models with polynomial non-linear features transformations.
 - Degree of polynomial depends on ‘n’.
 - Linear models with Gaussian RBFs as non-linear features transformations or kernels.
 - With RBF centered on each x^i .
- it can represent any function

Adding Bias Variables

- Recall fitting linear models with a **bias variable** (so $\hat{y}_i \neq 0$ when $x_i=0$).

Again, this is
representation in
new features.

$$\hat{y}_i = \sum_{j=1}^d w_j x_j + \beta$$

← adding one degree of freedom

- We often implement this by **adding a column of ones to X**.
- In neural networks we often include **biases on each z_{ic}** :

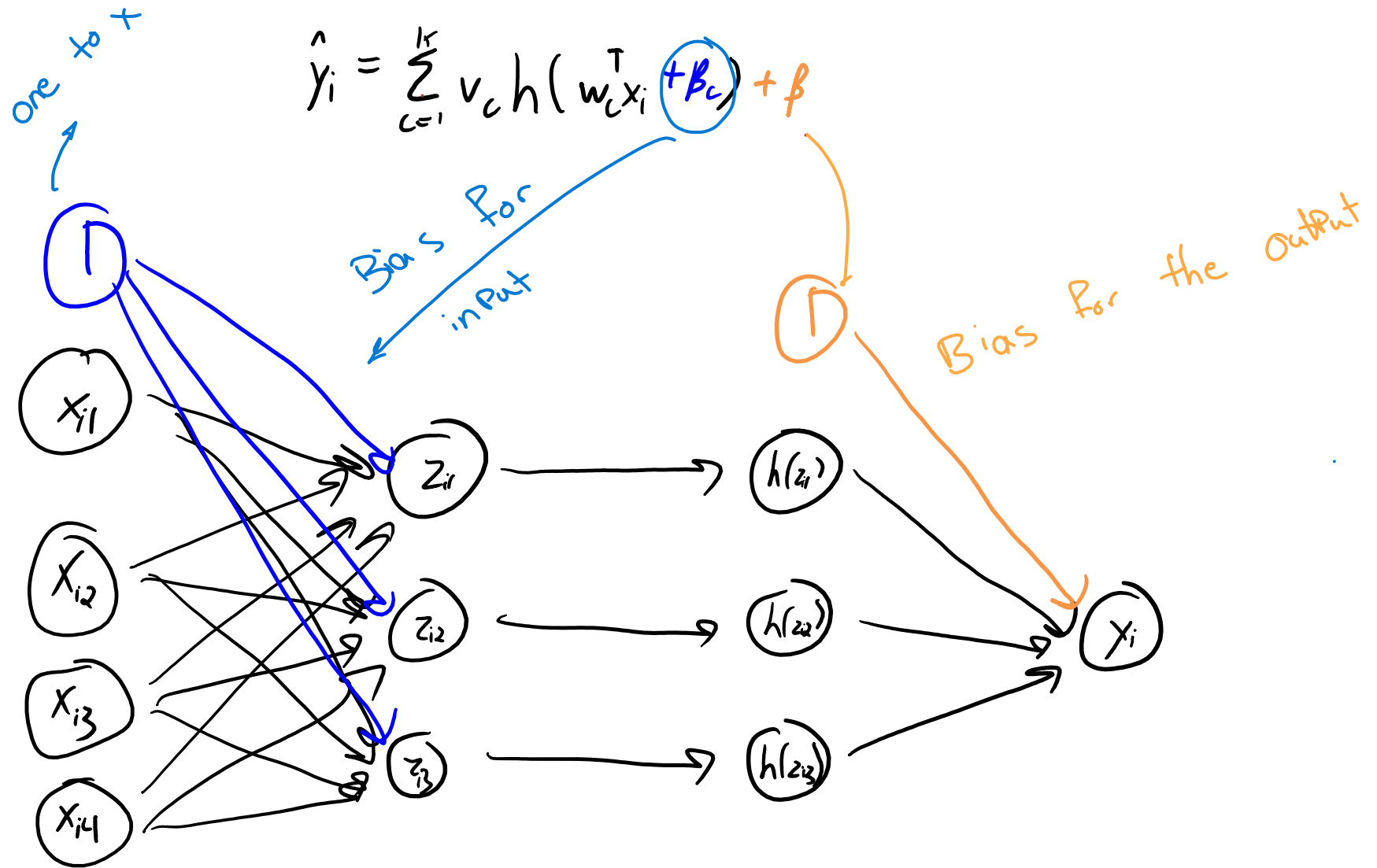
$$\hat{y}_i = \sum_{c=1}^K v_c h(w_c^T x_i + \beta_c)$$

- As before, we could implement this by **adding a column of ones to X**.
- We often also want a **bias on the output**:

$$\hat{y}_i = \sum_{c=1}^K v_c h(w_c^T x_i + \beta_c) + \beta$$

- For sigmoid 'h', you could equivalently fix **one row of W to be 0**.
 - Since $h(0)$ is a constant.

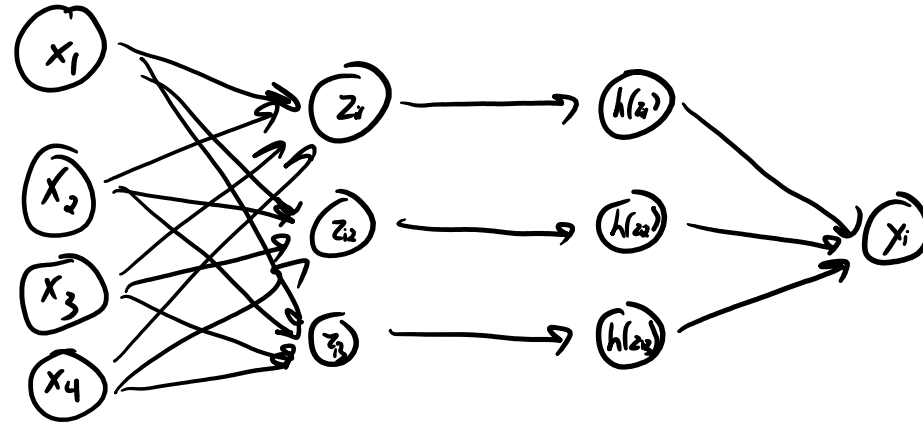
Adding Bias Variables



Regression vs. Binary Classification

- For **regression** problems, our prediction (ignoring biases) is:

$$\hat{y}_i = v^T h(W x_i)$$



- And we might train to minimize the **squared residual**:

$$f(W, v) = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{2} \sum_{i=1}^n (v^T h(W x_i) - y_i)^2$$

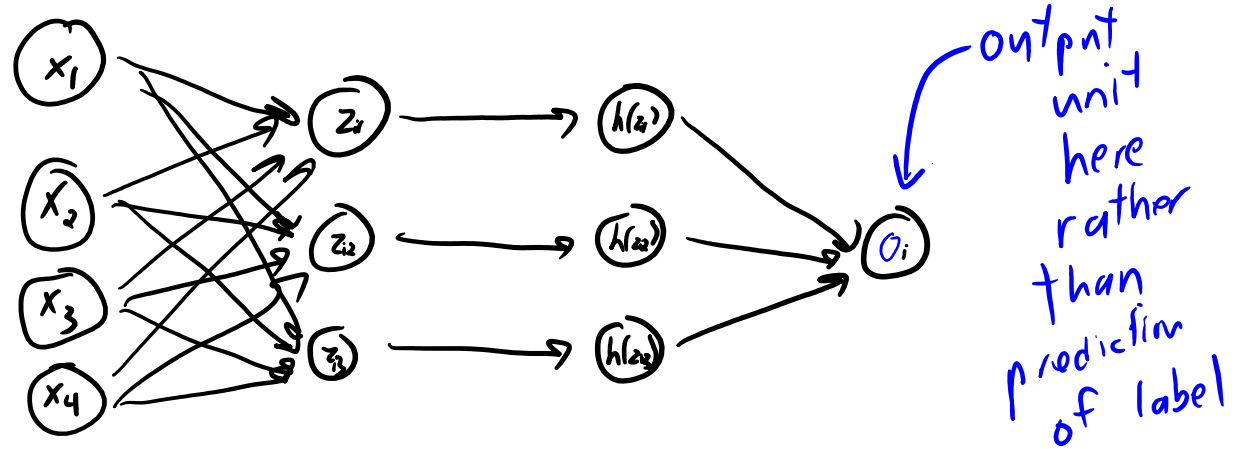
usually stochastic gradient descent.

Regression vs. Binary Classification

- For **binary classification** problems, our prediction is:

$$o_i = v^T h(Wx_i) \quad \hat{y}_i = \text{sign}(o_i)$$

Logistic Loss



- And we might train to minimize **the logistic loss**:

$$f(W, v) = \sum_{i=1}^n \log(1 + \exp(-y_i o_i)) = \sum_{i=1}^n \log(1 + \exp(-y_i v^T h(Wx_i)))$$

– This is like **logistic regression with learned features**.

$$p(y_i | W, v, x_i) = \frac{1}{1 + \exp(-y_i \underbrace{v^T h(Wx_i)}_{o_i})}$$

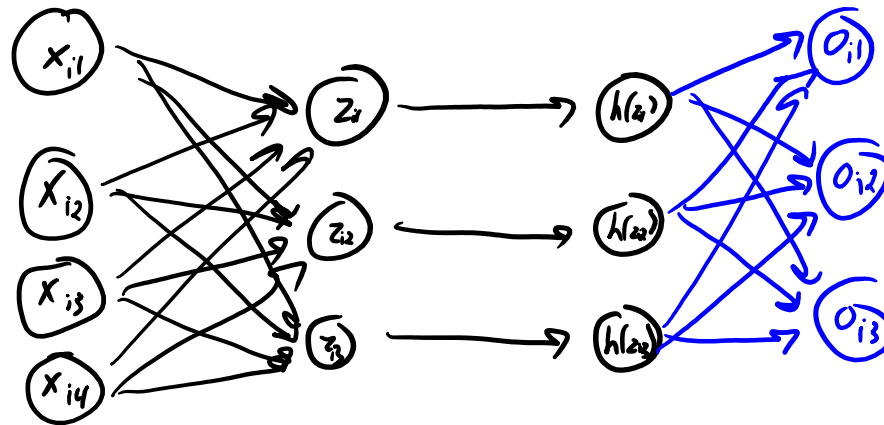
Use a Sigmoid on output to get a probability

Sigmoid σ

Neural Network for Multi-Class Classification

- Multi-class classification a **neural network**:

- Input is connected to a hidden layer (same as regression and binary case).
- Hidden layer is connected to multiple output units (one for each label.).



$$o_{i1} = v_1^T h(Wx_i)$$

$$o_{i2} = v_2^T h(Wx_i)$$

$$o_{i3} = v_3^T h(Wx_i)$$

only changing $\checkmark$

Now have a matrix of parameters:

$$V = \begin{bmatrix} v_1^T \\ v_2^T \\ \vdots \\ v_K^T \end{bmatrix}$$

$l \times K$
 (number of classes) $\times$ (number of hidden units)

- We can predict by maximizing o_{ic} over all 'c'.
- We can convert to probabilities for each class using softmax to the o_{ic} values:

$$\frac{\exp(o_{ic})}{\sum_{c'=1}^{K'} \exp(o_{ic'})} \rightarrow \text{softmax}$$

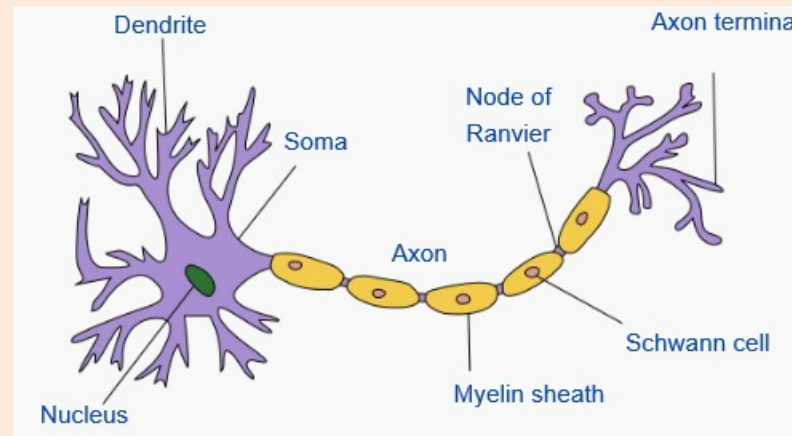
- We train by minimizing negative log of this probability (softmax loss, summed across examples).
- Notice that we changed tasks by only changing last layer (and loss function).

↓ softmax loss.

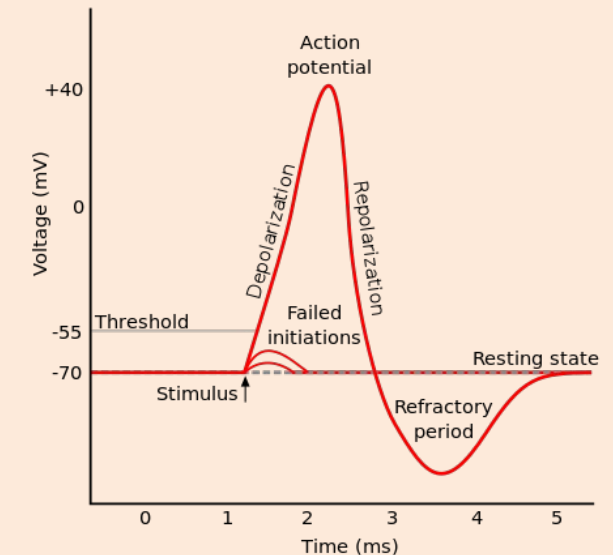
Next Topic: Biological Motivation

Why “Neural Network”?

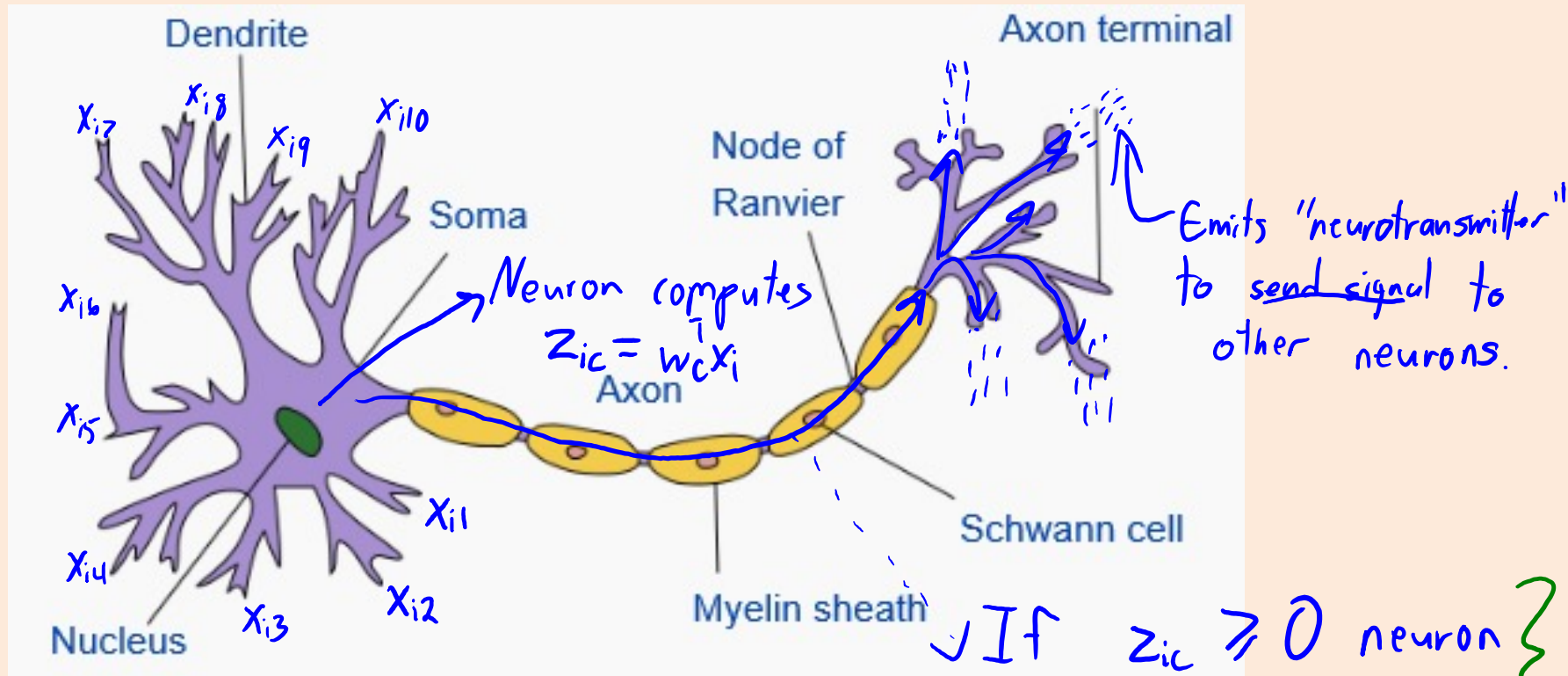
- Cartoon of “typical” neuron:



- Neuron has many “dendrites”, which take an input signal.
- Neuron has a single “axon”, which sends an output signal.
- With the right input to dendrites:
 - “Action potential” along axon (like a binary signal):

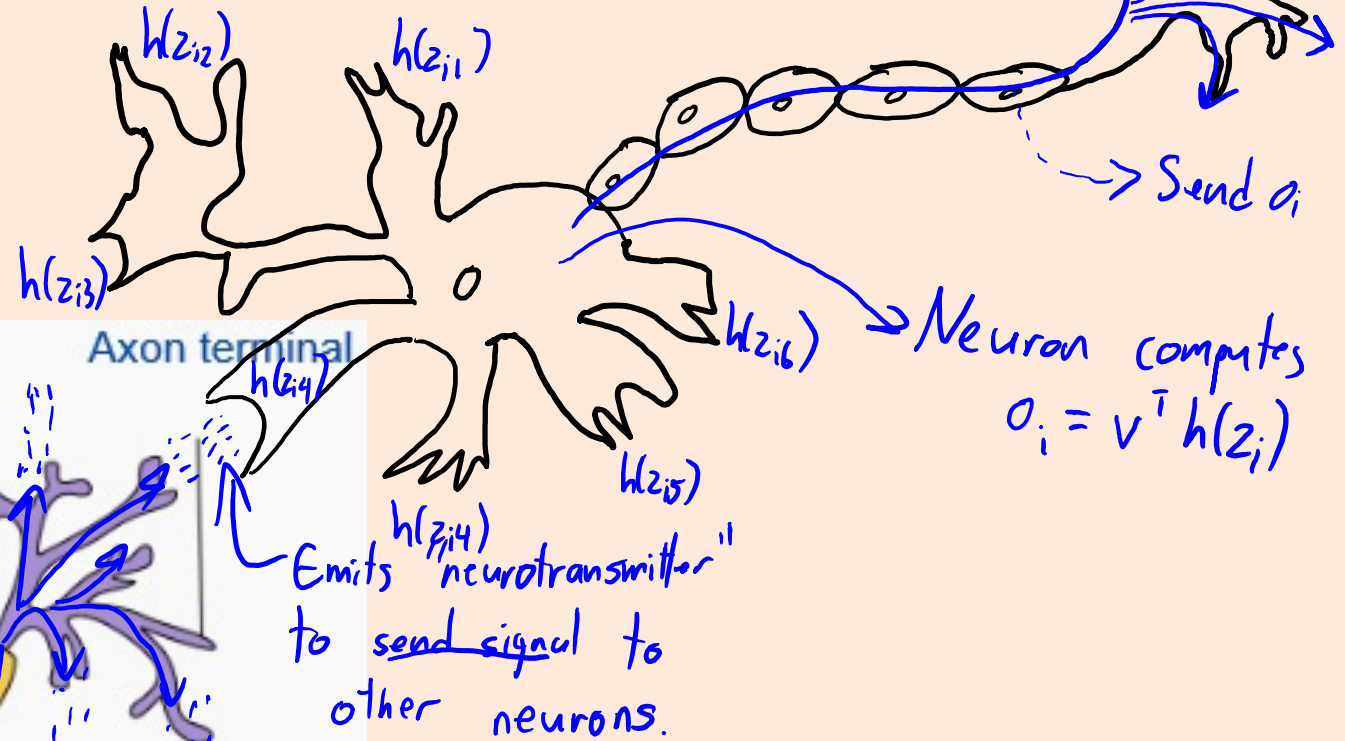
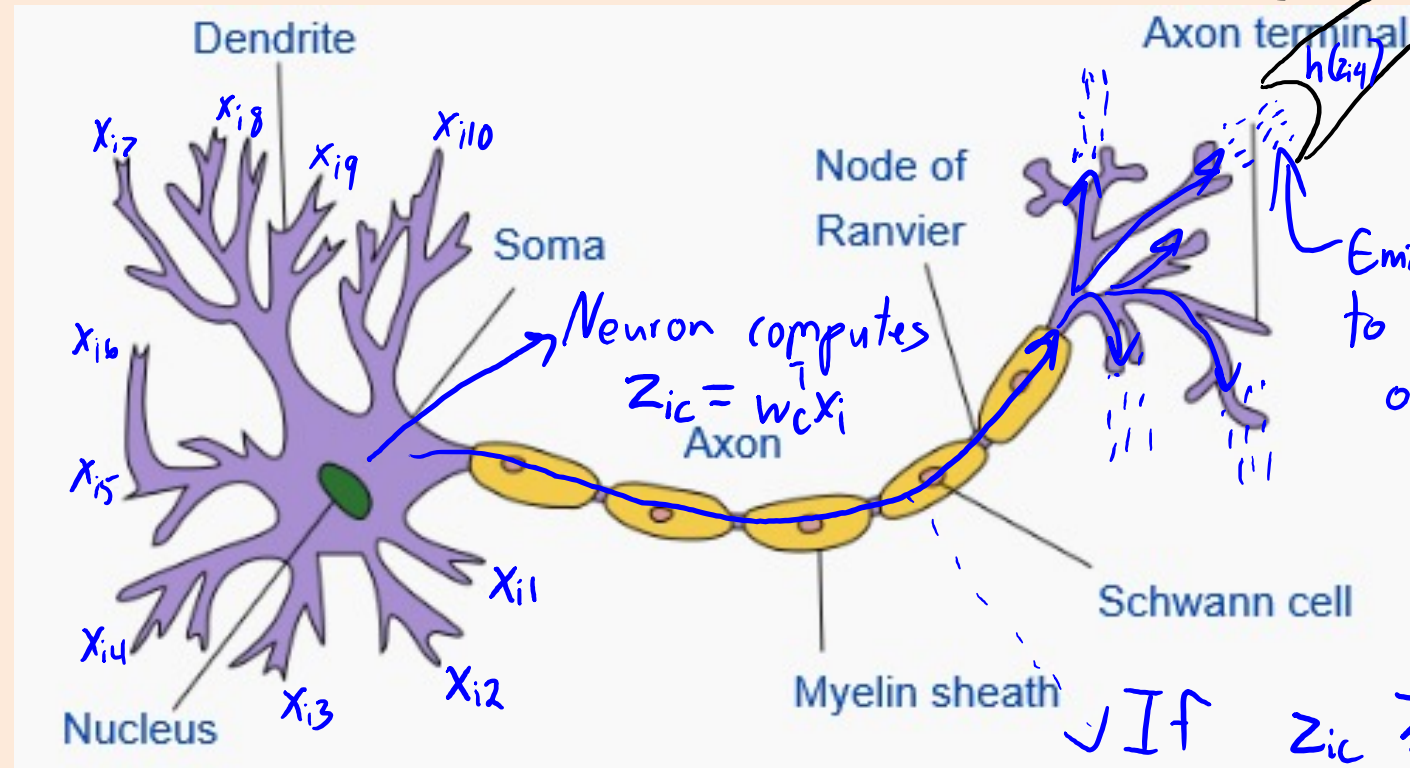


Why “Neural Network”?



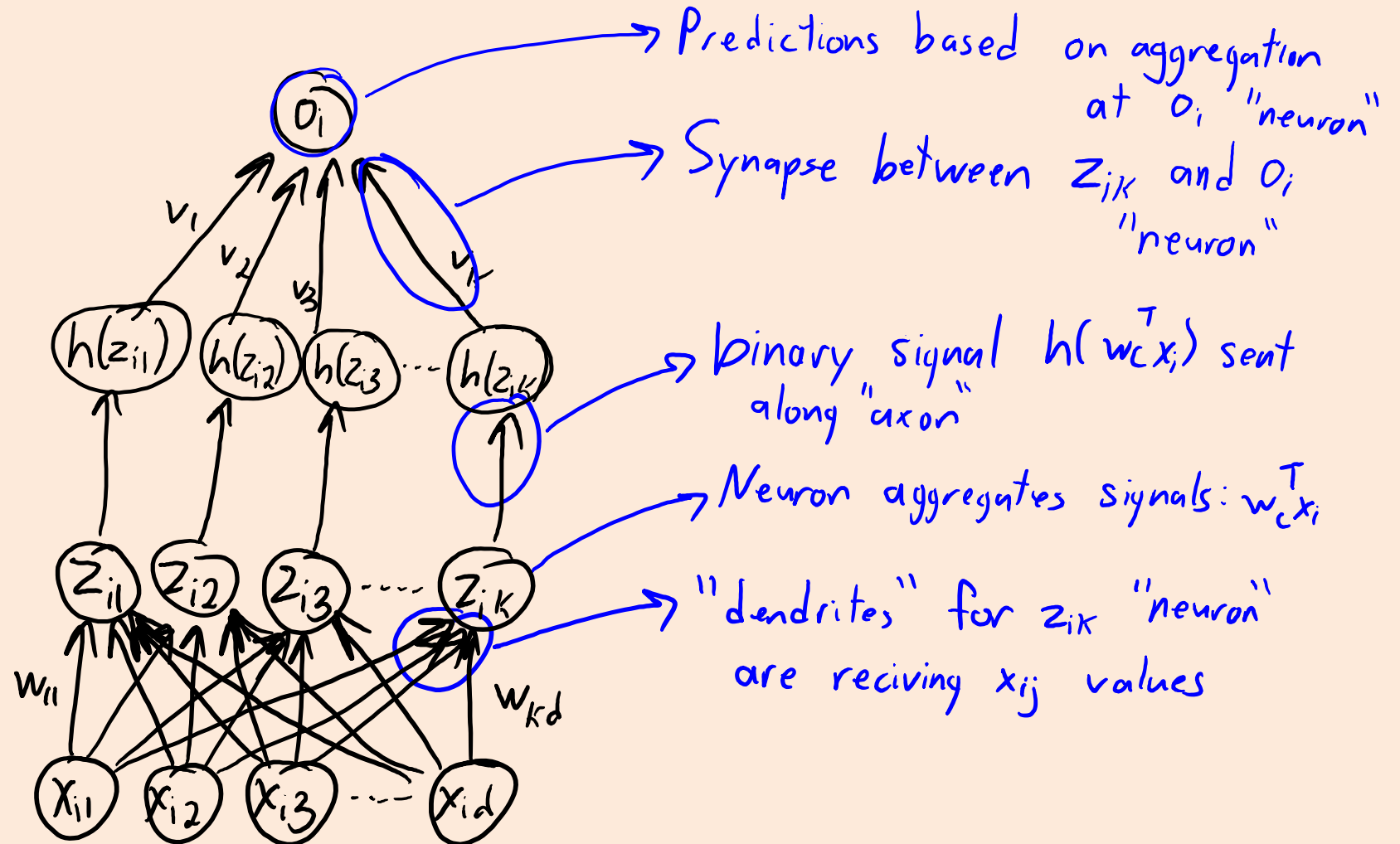
✓ If $z_{ic} \geq 0$ neuron } We approximate binary
Sends signal along axon. } signal with $\frac{1}{1 + \exp(-z_{ic})}$

Why "Neural Network"?



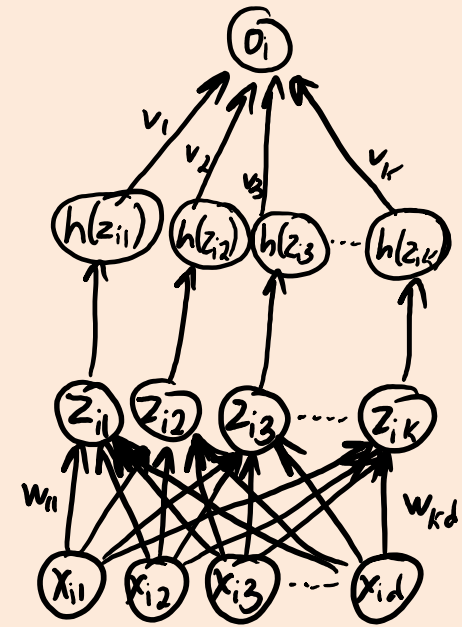
✓ If $z_{ic} \geq 0$ neuron } We approximate binary
 Sends signal along axon. } signal with $\frac{1}{1 + \exp(-z_{ic})}$

Why "Neural Network"?



“Artificial” Neural Nets vs. “Real” Networks Nets

- Artificial neural network:
 - x_i is measurement of the world.
 - z_i is internal representation of world.
 - o_i is output of neuron for classification/regression.
- Real neural networks are more complicated:
 - **Timing** of action potentials seems to be important.
 - “Rate coding”: frequency of action potentials simulates continuous output.
 - Neural networks don’t reflect **sparsity** of action potentials.
 - How much computation is done **inside neuron**?
 - Brain is highly **organized** (e.g., substructures and cortical columns).
 - Connection **structure changes**.
 - **Different types** of neurotransmitters.

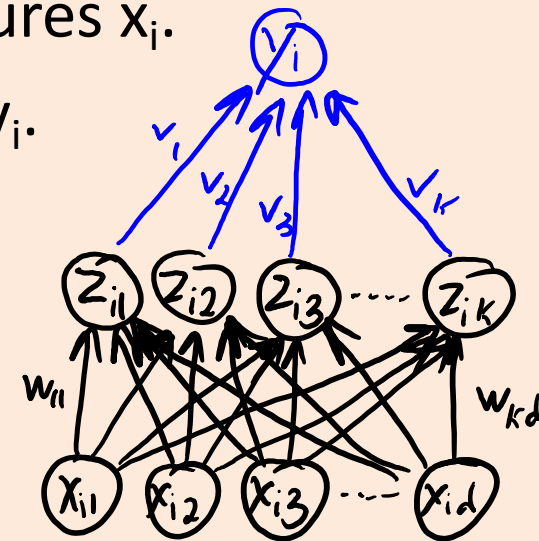


Summary

- Unprecedented performance on difficult pattern recognition tasks.
- Neural networks with one hidden layer:
 - Simultaneous learn a linear model and its features z_i .
- Non-linear transform avoids degeneracy.
 - Universal approximator if size of layer grows with number of examples 'n'.
- Bias variables added to each layer.
- Outputting probabilities and training with SGD.
- Biological motivation for (deep) neural networks. (Really ?)
- Next time: neural networks overfit less with more parameters?

Supervised Learning Roadmap

- Part 1: “Direct” **Supervised Learning**.
 - We learned parameters ‘ w ’ based on the **original features x_i** and target y_i .
- Part 3: **Change of Basis**.
 - We learned parameters ‘ v ’ based on a **change of basis z_i** and target y_i .
- Part 4: **Latent-Factor Models**.
 - We **learned parameters ‘ W ’ for basis z_i** based on only on features x_i .
 - You can **then learn ‘ v ’** based on change of basis z_i and target y_i .
- Part 5: **Neural Networks** (one hidden layer).
 - **Jointly learn ‘ W ’ and ‘ v ’ based on x_i and y_i .**
 - **Learn basis z_i that is good for supervised learning.**



Why $z_i = Wx_i$?

- In PCA we had that the optimal $Z = XW^T(WW^T)^{-1}$.
- If W had normalized+orthogonal rows, $Z = XW^T$ (since $WW^T = I$).
 - So $z_i = Wx_i$ in this normalized+orthogonal case.
- Why we would use $z_i = Wx_i$ in neural networks?
 - We didn't enforce normalization or orthogonality.
- Well, the value $W^T(WW^T)^{-1}$ is just “some matrix”.
 - You can think of neural networks as just **directly learning this matrix**.

Softmax NLL vs. Cross-Entropy

- Multi-class objective often written as minimizing **cross-entropy**:

$$f(W, V) = \sum_{i=1}^n \sum_{j=1}^L I[y^i = c] (-\log p(y^i = c | X, W, V))$$

- The indicator function is **zero except for true label y^i** :

$$f(W, V) = - \sum_{i=1}^n \log p(y^i | X, W, V)$$

- When we plug in the softmax likelihood, we get the **softmax NLL**.
 - So **cross-entropy is the softmax NLL** with extra terms that do nothing.
 - Cross-entropy way of writing would make more sense if training data had “soft” assignments to classes.